

Recursos ISIS-2211

Angular Signals

Teoría

Reactividad sin magia: qué son los Signals de Angular, cómo funcionan y cuándo usar cada una de sus tres variantes.

¿Qué es un Signal?

Un **Signal** es una variable reactiva. La diferencia clave frente a una variable normal es esta: cuando su valor cambia, Angular actualiza **automáticamente** cualquier parte del template que lo use. No tiene que decirle a Angular "oye, actualiza esto", él lo detecta solo.

Los Signals están disponibles en Angular **desde la versión 17**.

Analogía

Una variable normal es como escribir algo en papel: si lo borras y reescribes, nadie se entera. Un Signal es como un pizarrón frente a la clase: en cuanto el valor cambia, todos los que están mirando lo ven de inmediato.

X Variable normal

```
count = 0;

// Para actualizar el template
// Angular no se entera solo.
// Dependes de zone.js
// o markForCheck().
count++;
```

✓ Signal

```
count = signal(0);

// Angular detecta el cambio
// y actualiza el template
// automáticamente.
count.set(count() + 1);
```

Los tres bloques

Con estos tres conceptos ya puede escribir componentes reactivos completos.

signal(), Signal base

Es el punto de partida. Crea un valor reactivo que puede cambiar. Para **leer**: `count()`. Para **escribir**: `count.set(n)` o `count.update(v => v+1)`.

computed(), Signal derivado

Solo lectura. Su valor se calcula a partir de otros Signals. Produce un **valor** que puede mostrar en el template. Se recalcula automáticamente y está memorizado.

effect(), Efecto secundario

No produce ningún valor. Ejecuta **código** cada vez que un Signal cambia: guardar en `localStorage`, llamar una API, sincronizar con librerías externas.



Sintaxis en un componente

Este es el patrón completo que verá en su primer componente Angular con Signals:

counter.component.ts

```
import { Component, signal, computed, effect } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <p>Contador: {{ count() }}</p>
    <p>El doble: {{ double() }}</p>
    <button (click)="incrementar()">+1</button>
  `,
})
export class CounterComponent {

  // 1. Signal base, valor que puede cambiar
  count = signal(0);

  // 2. computed, solo lectura, produce un valor para el template
  double = computed(() => this.count() * 2);

  // 3. effect, ejecuta código cuando count cambia
  constructor() {
    effect(() => {
      console.log('Nuevo valor:', this.count());
    });
  }

  incrementar() {
    this.count.update(v => v + 1); // equivalente a .set(count() + 1)
  }
}
```

⚠ ¿Por qué se ve igual que una variable normal?

En este ejemplo el mismo componente escribe (`incrementar()`) y lee (`{{ count() }}`) el valor, y las dos cosas pasan en el mismo clic. Con `zone.js` — el modo por defecto de Angular — ese clic ya dispara una revisión de todo el árbol de componentes, así que una variable normal (`count++`) también se habría actualizado en pantalla. Acá no hay ninguna diferencia visible, y está bien notarlo en vez de fingir que sí la hay.

La diferencia aparece cuando quien escribe y quien lee son **componentes distintos**, sin `@Input/@Output` ni suscripciones entre ellos. Más abajo, después de practicar la sintaxis, hay un demo que separa esos dos roles.

Acción	Sintaxis	Qué hace
Leer	<code>count()</code>	Llama al Signal como función
Escribir	<code>count.set(n)</code>	Reemplaza el valor completo
Actualizar	<code>count.update(v => v+1)</code>	Calcula a partir del valor actual
Derivar	<code>computed(() => ...)</code>	Solo lectura, memorizado

Pruébelo usted mismo [🔗](#)

Interactúa con cada concepto para ver cómo se comporta en tiempo real.

`signal()` `computed()` `effect()`

```
count = signal(0); // crear
count();           // leer → 0
count.set(5);      // escribir
```

```
count() = 0
```

- restar 1

+ sumar 1

reset a 0

Cada vez que llamas a `.set()`, Angular detecta el cambio y actualiza el template automáticamente. Sin `ChangeDetectorRef`, sin `markForCheck()`.

Signals más allá de un componente

Recuerde la nota de más arriba: en el contador simple, quien escribe y quien lee el valor es el mismo componente, así que no había ninguna diferencia frente a una variable normal. Este demo separa esos dos roles en dos componentes que **no se conocen entre sí** — la única conexión es un servicio:

Clase	Qué es	Qué hace
CounterService	Servicio (@Injectable)	Es dueño de la signal <code>count</code> y de las derivadas <code>doubled</code> , <code>isEven</code> , <code>label</code> . Expone los métodos (<code>increment</code> , <code>decrement</code> , <code>reset</code>) para cambiarla.
ControlsComponent	Componente	Injecta <code>CounterService</code> con <code>inject()</code> y llama sus métodos. No guarda ni muestra ningún valor.
DisplayComponent	Componente	Injecta <code>CounterService</code> con <code>inject()</code> y lee sus signals en el template. No modifica nada.

“Injectar” un servicio no es crear una instancia propia. `inject(CounterService)` le pide a Angular la instancia de esa clase — como `CounterService` está registrado con

providedIn: 'root', Angular crea **una sola** instancia para toda la aplicación y se la entrega a quien la pida. Por eso ControlsComponent y DisplayComponent terminan compartiendo exactamente la misma instancia, con las mismas signals adentro, sin que uno sepa que el otro existe.

counter.service.ts

```
@Injectable({ providedIn: 'root' })
export class CounterService {
  private _count = signal(0);
  readonly count = this._count.asReadonly();

  readonly doubled = computed(() => this._count() * 2);
  readonly isEven = computed(() => this._count() % 2 === 0);
  readonly label = computed(() => `Contador: ${this._count()}`);

  increment() { this._count.update(v => v + 1); }
  decrement() { this._count.update(v => v - 1); }
  reset() { this._count.set(0); }
}
```

controls.component.ts

```
@Component({
  selector: 'app-controls',
  template: `
    <button (click)="service.decrement()">- 1</button>
    <button (click)="service.increment()">+ 1</button>
    <button (click)="service.reset()">reset</button>
  `,
})
export class ControlsComponent {
  service = inject(CounterService);
}
```

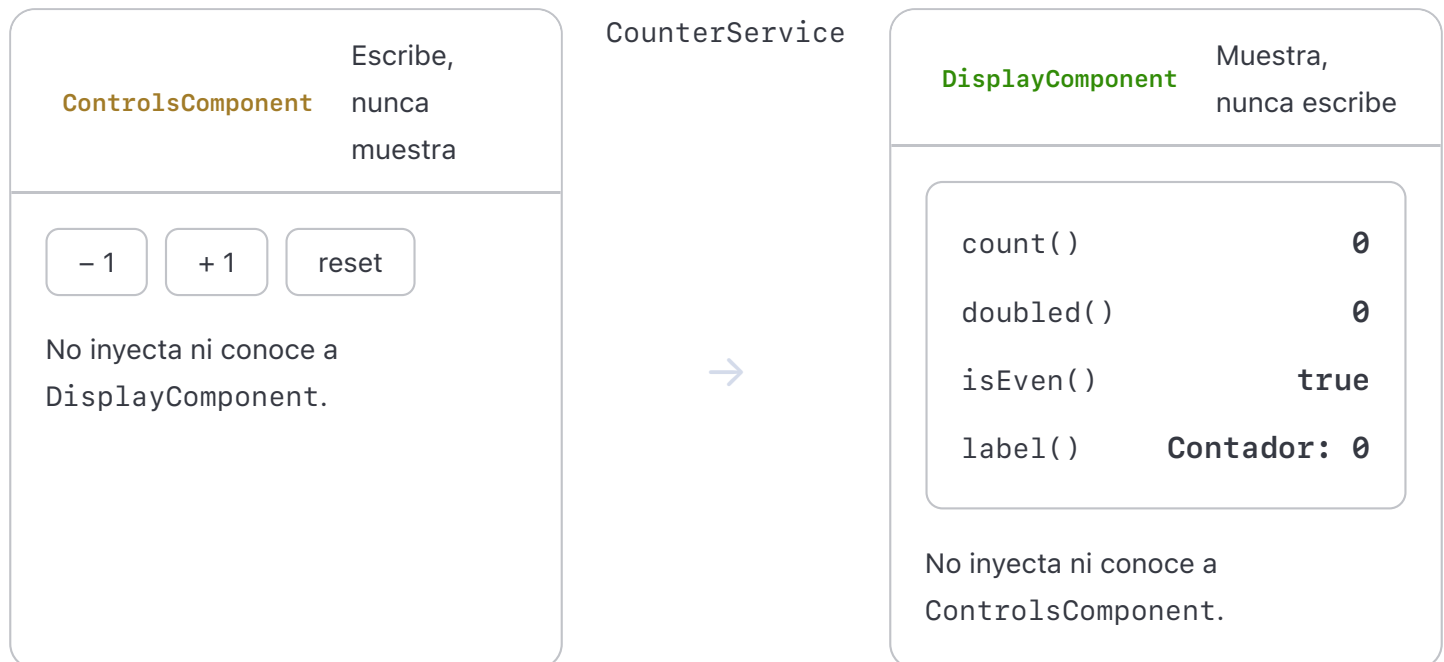
display.component.ts

```

@Component({
  selector: 'app-display',
  template: `
    <p>count: {{ service.count() }}</p>
    <p>doubled: {{ service.doubled() }}</p>
    <p>isEven: {{ service.isEven() }}</p>
    <p>{{ service.label() }}</p>
  `,
})
export class DisplayComponent {
  service = inject(CounterService);
}

```

Así se ve en la práctica — pruebe hacer clic a la izquierda y observe la derecha:



Ningún componente conoce al otro. No hay @Input, no hay @Output, no hay suscripción manual. La única conexión es CounterService: cuando ControlsComponent llama increment(), la signal cambia dentro del servicio, y DisplayComponent se actualiza solo porque lee esa signal en su template.

computed() vs effect()

La pregunta que le resuelve la duda en un segundo: **¿necesita un valor para mostrar en el template?** Si sí → `computed()`. Si lo que necesita es que algo *ocurra* cuando un Signal cambia → `effect()`.

Analogía

computed() es como una celda de Excel con fórmula: cuando cambia una celda de la que depende, el resultado se actualiza solo y lo ves en pantalla.

effect() es como una macro que se dispara cuando cambia esa celda: no produce un valor visible, sino que *hace algo*, manda un correo, guarda un archivo, actualiza otro sistema.

El siguiente demo usa un carrito de compras. A la izquierda, `computed()` calcula valores que se **muestran** en pantalla. A la derecha, `effect()` **ejecuta acciones** cada vez que el carrito cambia, sin producir ningún valor visible por sí mismo.

computed() Produce valores para el template

Teclado mecánico \$85.000

– 0 +

Mouse inalámbrico \$45.000

– 0 +

Monitor 24" \$420.000

– 0 +

itemCount() 0 items

hayItems() false

effect() Ejecuta acciones secundarias

// Se dispara cada vez que el
carrito cambia

 Componente inicializado — `effect()`
se ejecutó por primera vez

total() \$0

¿Necesitas mostrar algo en el template?

→ **usa computed()**

total(), hayItems(),
precioFormateado()

¿Necesitas que algo ocurra cuando un Signal cambia?

→ **usa effect()**

guardar en localStorage, llamar API,
loggear

¿Cuándo usar cada uno?

La elección entre los tres depende de si el valor es **origen**, **derivado** o **acción**:

Regla de decisión

```
// ¿El valor cambia por acción del usuario o del sistema?  
// → signal()  
nombre      = signal('');  
isLoading   = signal(false);  
productos   = signal<Producto[]>([]);  
  
// ¿El valor se calcula a partir de otros Signals y lo muestra en el  
// template?  
// → computed()  
total       = computed(() => this.productos().reduce((s, p) => s + p.precio,  
0));  
hayItems    = computed(() => this.productos().length > 0);  
  
// ¿Necesita que algo ocurra cuando un Signal cambia?  
// → effect()  
effect(() => {  
  localStorage.setItem('nombre', this.nombre());  
});
```

Autoevaluación

1. ¿Qué pasa cuando cambia el valor de un Signal?

- ☐ Angular actualiza automáticamente cualquier parte del template que lo use
- ☐ Hay que llamar manualmente a `markForCheck()` para que se refleje
- ☐ No pasa nada hasta que el usuario recarga la página

2. ¿Cómo se lee el valor actual de un Signal llamado count?

- ☐ `count.value`
- ☐ `count()`
- ☐ `count.get()`

3. ¿Cuál es la diferencia entre `.set()` y `.update()`?

- ☐ `.set()` reemplaza el valor completo; `.update()` lo calcula a partir del valor actual
- ☐ Son dos nombres distintos para exactamente lo mismo
- ☐ `.update()` solo existe en `computed()`, no en `signal()`

4. ¿Qué produce `computed()`?

- ☐ Un valor derivado de solo lectura, memorizado y recalculado automáticamente
- ☐ Un valor que se puede modificar llamando a `.set()`
- ☐ No produce ningún valor, solo ejecuta código

5. ¿Qué diferencia a `effect()` de `computed()`?

¿Qué diferencia a effect() de computed()?

- ☐ effect() no produce ningún valor: ejecuta código como efecto secundario cuando un Signal cambia
- ☐ effect() es la única forma de leer el valor de un Signal
- ☐ effect() reemplaza a signal() en las versiones más recientes de Angular

6. Necesita un valor para mostrar en el template a partir de otros Signals. ¿Cuál usa?

- ☐ computed()
- ☐ effect()
- ☐ signal.update()

7. CounterService está registrado con providedIn: 'root'. ¿Qué pasa cuando dos componentes distintos hacen inject(CounterService)?

- ☐ Angular crea una instancia nueva para cada componente que lo inyecta
- ☐ Angular entrega la misma instancia única a ambos, por eso comparten las mismas signals
- ☐ Angular no permite inyectar un servicio que contenga signals